

HCI & CG LAB REPORT

ALI AHMED (2K24/CSE/25)

Task 1: Display Pixel Density (PPI/DPI) Calculator.

Python sources Code:

```
import math

# Input
horizontal_pixels = int(input("Enter horizontal resolution (pixels): "))
vertical_pixels = int(input("Enter vertical resolution (pixels): "))
diagonal = float(input("Enter physical diagonal size (inches): "))

# Total pixel count
total_pixels = horizontal_pixels * vertical_pixels

# Aspect ratio
gcd = math.gcd(horizontal_pixels, vertical_pixels)
aspect_width = horizontal_pixels // gcd
aspect_height = vertical_pixels // gcd

# Calculate DPI
dpi = math.sqrt(horizontal_pixels**2 + vertical_pixels**2) / diagonal

# Display results
print("\n--- DISPLAY METRICS ANALYSIS ---")
print(f"Total Pixel Count: {total_pixels:,}")
print(f"Aspect Ratio: {aspect_width}:{aspect_height}")
print(f"Calculated DPI: {dpi:.2f}" " DPI")
```

```
# Density classification

if dpi < 100:

    density = "Low density (Standard Monitor)"

elif dpi <= 200:

    density = "Medium density (HD display)"

else:

    density = "High density (Retina mobile)"

print(f"Density Category: {density}")
```

Output:

```
PS D:\Confidential Data\2. CSE\3rd year\6th Semester\HCI & Computer Graphics\hci Ass> & "C:\Users\Ali Ahmed\
"d:/Confidential Data/2. CSE/3rd year/6th Semester/HCI & Computer Graphics/hci Ass/1_pixel_density_cal.py"
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count: 2,073,600
Aspect Ratio: 16:9
Calculated DPI: 91.79 DPI
Density Category: Low density (Standard Monitor)
```

```
PS D:\Confidential Data\2. CSE\3rd year\6th Semester\HCI & Computer Graphics\hci Ass> & "C:\Users\Ali Ahmed\
"d:/Confidential Data/2. CSE/3rd year/6th Semester/HCI & Computer Graphics/hci Ass/1_pixel_density_cal.py"
Enter horizontal resolution (pixels): 1170
Enter vertical resolution (pixels): 2532
Enter physical diagonal size (inches): 6.1

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count: 2,962,440
Aspect Ratio: 195:422
Calculated DPI: 457.25 DPI
Density Category: High density (Retina mobile)
```

Task 2: Environment Setup & Synthetic Image Matrix Creation

Python sources Code:

```
import numpy as np

from PIL import Image

import matplotlib.pyplot as plt

# Create a 300 x 400 x 3 image matrix filled with zeros

image = np.zeros((300, 400, 3), dtype=np.uint8)
```

```
# Get the middle points
row_end = 300
col_end = 400
row_mid = row_end // 2
col_mid = col_end // 2

# Top-left: Red
image[0:row_mid, 0:col_mid] = [255, 0, 0]

# Top-right: Green
image[0:row_mid, col_mid:col_end] = [0, 255, 0]

# Bottom-left: Blue
image[row_mid:row_end, 0:col_mid] = [0, 0, 255]

# Bottom-right: White
image[row_mid:row_end, col_mid:col_end] = [255, 255, 255]

# Display matrix information
print("\n--- SYNTHETIC METRIX METRICS ---")
print("Array Shape (H, W, C) :", image.shape)
print("Data Type :", image.dtype)
print("Total Elements :", image.size, "Values")
print("Memory Footprint :", image.nbytes, "bytes (351.56 KB)")

# Display the image
plt.imshow(image)
plt.axis("off")
plt.show()
```

OUTPUT

```
PS D:\Confidential Data\2. CSE\3rd year\6th Semester\HCI & Computer Graphics\hci Ass> & "C:\Users\Ali Ahmed\
"d:\Confidential Data\2. CSE\3rd year\6th Semester\HCI & Computer Graphics\hci Ass\2_Synthetic_image_matrix

--- SYNTHETIC METRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type : uint8
Total Elements : 360000 Values
Memory Footprint : 360000 bytes (351.56 KB)
█
```



Task 3: Channel Slicing & Isolation

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from PIL import Image
```

```
import os
```

```
# 1. Find sample.jpg in the same folder as this Python file
```

```
script_dir = os.path.dirname(os.path.abspath(__file__))
```

```
image_path = os.path.join(script_dir, "sample.jpg")
```

```
# Load input image
```

```
img = np.array(Image.open(image_path))
```

```
# 2. Extract 2D channel intensity grids using Axis 2 slicing
```

```
red = img[:, :, 0]
```

```
green = img[:, :, 1]
```

```
blue = img[:, :, 2]
```

```
# 3. Create isolated 3D color arrays
```

```
red_only = np.zeros_like(img)
```

```
green_only = np.zeros_like(img)
```

```
blue_only = np.zeros_like(img)
```

```
red_only[:, :, 0] = red
```

```
green_only[:, :, 1] = green
```

```
blue_only[:, :, 2] = blue
```

```
# 4. Channel Extraction Summary
```

```
print("--- CHANNEL EXTRACTION SUMMARY ---")
```

```
print(f"Original Image Shape : {img.shape}")
```

```
print(f"Red Channel 2D Shape : {red.shape} | Mean Intensity: {red.mean():.2f}")
```

```
print(f"Green Channel 2D Shape: {green.shape} | Mean Intensity: {green.mean():.2f}")
```

```
print(f"Blue Channel 2D Shape : {blue.shape} | Mean Intensity: {blue.mean():.2f}")
```

```
# 5. Display 2 × 3 subplot
```

```
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
```

```
# Top row - isolated color images
```

```
axes[0, 0].imshow(red_only)
```

```
axes[0, 0].set_title("Red-Only")
```

```
axes[0, 1].imshow(green_only)
```

```
axes[0, 1].set_title("Green-Only")
```

```
axes[0, 2].imshow(blue_only)
```

```
axes[0, 2].set_title("Blue-Only")
```

```
# Bottom row - grayscale intensity maps
```

```
axes[1, 0].imshow(red, cmap="gray")
```

```
axes[1, 0].set_title("Red Intensity")
```

```
axes[1, 1].imshow(green, cmap="gray")
```

```
axes[1, 1].set_title("Green Intensity")
```

```
axes[1, 2].imshow(blue, cmap="gray")
```

```
axes[1, 2].set_title("Blue Intensity")
```

```
# Hide axes
```

```
for ax in axes.flat:
```

```
    ax.axis("off")
```

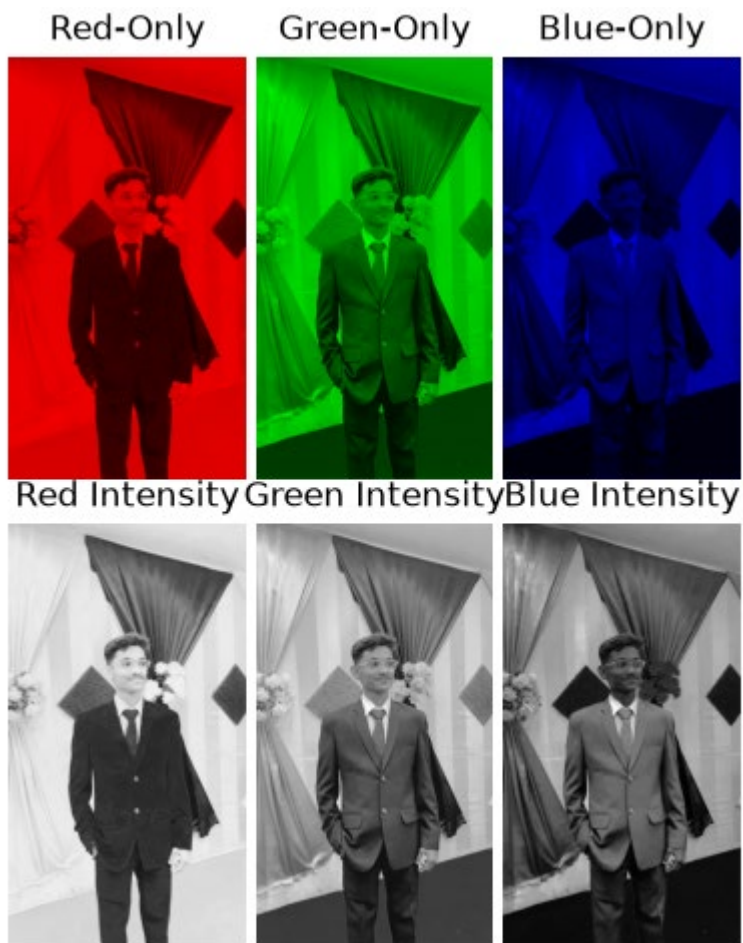
```
plt.tight_layout()
```

```
print("Display Window : Matplotlib 2x3 Subplot Grid Rendered.")
```

```
plt.show()
```

OUTPUT

```
PS D:\Confidential Data\2. CSE\3rd year\6th Semester\HCI & C
"d:/Confidential Data/2. CSE/3rd year/6th Semester/HCI & Co
--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (1334, 750, 3)
Red Channel 2D Shape : (1334, 750) | Mean Intensity: 164.98
Green Channel 2D Shape: (1334, 750) | Mean Intensity: 127.71
Blue Channel 2D Shape : (1334, 750) | Mean Intensity: 94.53
Display Window : Matplotlib 2x3 Subplot Grid Rendered.
█
```



Q: Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

Ans: Because RGB uses 3 bytes per pixel (Red, Green, Blue), while RGBA uses 4 bytes per pixel (adding Alpha/transparency).

So memory increases by:

$$\frac{4 - 3}{3} \times 100\% = 33.3\%$$

In short: the extra Alpha channel adds 1 byte for every 3 bytes previously used.